1 Stochastic Model and Quality Control

This document covers the *measurement* noise of the estimator. It describes the strategies that fill the observation-noise covariance $\boldsymbol{R}$, and the a-posteriori quality-control (QC) layer that tests an epoch's residuals against that $\boldsymbol{R}$. The $\boldsymbol{R}$ here has the block-diagonal structure of the observation-equations noise model, and this document gives the variance each strategy puts in it. The QC layer uses the post-fit residual statistics that the filter already computes for outlier rejection, and presents them in the classical geodetic detection theory of Baarda and Teunissen [7].

Two topics are outside this document and are only cross-referenced. The first is the reaction to a detected blunder: the standardised-residual outlier path ($|s| > 6$), the Huber re-weighting, and the EWMA inflation of $\boldsymbol{R}$. These are in the Kalman-filter gating section. This document explains how the QC statistics drive an *alternative* gate but does not repeat the dispatch. The second is the process-noise covariance $\boldsymbol{Q}$ (clock random walk, ZWD, ambiguity, IMU bias), which belongs to the dynamics and is described with the smoother (the process-noise section). Everything below is measurement noise.

1.1 The stochastic-model dispatch

Every undifferenced observation (r, i, s) is assigned a scalar variance by a strategy object selected from the `StochasticModelType` enum (`include/config/GnssConfigTypes.hpp`, the four-way {`Constant`, `Elevation`, `SNR`, `SatelliteBlock`} set). The dispatch is in `src/core/model/StochasticModel.cpp`. Each strategy returns a variance in m^2, which becomes one diagonal entry of $\boldsymbol{R}$. The three strategies that use geometry share a low-elevation guard. When $\sin\theta \leq 10^{-6}$ the elevation amplification is skipped and the unamplified variance is returned, so a satellite at the horizon does not receive a numerically infinite variance (`computeElevation`). The `Constant` strategy has no elevation term. It returns its a-priori variance directly (`computeConstant`) and never reaches the guard.

(a) Constant. This strategy returns the a-priori variance of unit weight unchanged,

$$\sigma^2_{r,i,s} = \sigma^2_{0,\mathrm{type}(s)}, \tag{1}$$

with σ_0 drawn from `filter.measurement_noise` (`computeConstant`). The other strategies scale this value.

(b) Elevation. The classical elevation-dependent model scales the zenith-referenced variance by an inverse power of $\sin\theta$,

$$\boxed{\sigma^2_{r,i,s} = \sigma^2_{\mathrm{zen}} \left(\sin\theta^i_r\right)^{-2p_{\mathrm{type}}}, \qquad p_P = p_\Phi = 1.0.} \tag{2}$$

The shipped defaults of `filter.code_elevation_power` and `filter.phase_elevation_power` are both one, so the shipped model is *symmetric*: code and phase both follow the legacy $\csc^2\theta$ variance law (`computeElevation`). The exponent is $-2p_{\mathrm{type}}$ because the strategy works in variance, not σ: $\left(\sin\theta\right)^{-2p} = \left(\sin\theta^{-p}\right)^2$. Rows whose observable code does not begin with `L` take the code power. That is every row except carrier phase, including Doppler and the fallback paths with an empty `obs_type`.

Both exponents are configurable per observable class. The asymmetric setting $p_\Phi = 0.5$ is the one alternative with a physical argument, so the reasons it is an *option* and not the default are recorded here. It gives a $\csc\theta$ variance ($\csc^{1/2}\theta$ in σ), which amplifies much less at low elevation.

The argument for it has two parts. Phase noise near the horizon is dominated by carrier-loop SNR and by antenna phase-centre miscalibration, and neither follows the $\csc^2\theta$ law that governs code multipath and pseudorange path-length error. The milder amplification also keeps low-elevation phase within the useful range of the standardised-residual cycle-slip test, where a steeper law would deweight it until the test has no effect.

The argument against it is comparability: the code branch and every archived benchmark run at one. The loader therefore ships one (`GnssConfig::getPhaseElevationPower`, whose documentation gives the amplification ratios at a 10° mask).

(c) SNR (SIGMA-ε). When carrier-to-noise density is available the strategy uses the Hadás–Bosy variance-additive form [4]. Writing the measured (or synthesised) density as $\mathrm{C/N_0}$ in dB Hz,

$$\boxed{\sigma^2_{\mathrm{SNR}} = a^2 + b^2\, 10^{-\mathrm{C/N_0}/10},} \tag{3}$$

where a is the high-SNR floor ($\sigma \to a$ as $\mathrm{C/N_0} \to \infty$) and b scales the SNR-dependent contribution; both have separate code/phase values (`snr_params_.code_a/b`, `phase_a/b`). Adding *variances*, as in Equation (3), is the correct way to combine two independent noise sources. The legacy alternative adds standard deviations, $\sigma = a + b\,10^{-\mathrm{C/N_0}/10}$. `computeSnr` rejects it, because squaring it produces a cross-term $2ab\,10^{-\mathrm{C/N_0}/10}$ that corresponds to no noise process: it treats the two independent contributions as correlated.

When no `S...` observable is present the model keeps the same functional form. It *synthesises* a density from elevation, so the noise curve stays continuous (`synthesizeCn0FromElevation`):

$$\mathrm{C/N_0}(\theta) = 50 - 15\,\big(1 - \max(0, \sin\theta)\big) \ \ [\mathrm{dB\,Hz}], \tag{4}$$

which is a maximum of 50 dB Hz at zenith, falling by 15 dB at the horizon. The density is then clamped to the configured working range [`snr_floor_dbhz`, `snr_ceil_dbhz`]. Two refinements follow.

On code rows, a per-band scale s^2_{band} from the SatelliteBlock code table multiplies σ^2_{SNR}. Wider-bandwidth signals such as L5/E5a have $\sim\sqrt{10}$ less correlator noise in σ, a factor of $\sim 1/10$ in variance. Phase has no band scale, because PLL noise depends on the carrier-loop bandwidth, not the chip rate.

The Wieser hybrid step then passes the SNR variance through Equation (2). Pure SNR weighting under-weights multipath that is correlated with elevation, because the carrier loop can track cleanly while the range is corrupted. The elevation factor accounts for it.

(d) SatelliteBlock. The most detailed strategy keys the zenith σ on the transmitting satellite's ANTEX block type, then applies observable-specific frequency-band and tracking-attribute scales (`computeSatBlock`):

$$\boxed{\sigma^2_{r,i,s} = \big(\sigma_{\mathrm{block}(i),\,\mathrm{type}}\; s_{\mathrm{band}}\; s_{\mathrm{attr}}\big)^2 \big(\sin\theta\big)^{-2p_{\mathrm{type}}}.} \tag{5}$$

Here $\sigma_{\mathrm{block}(i),\mathrm{type}}$ is looked up by substring-matching the satellite's ATX type string against `block_code_std_m` / `block_phase_std_m` (falling back to `default_code_std_m` / `default_phase_std_m` when no block matches); s_{band} is the per-band factor keyed on the RINEX band character (`obs_type[1]`); and s_{attr} is the tracking-attribute factor keyed on `obs_type[2]` via a *per-system* table, because a RINEX-3 attribute means different things in different constellations: a `C` attribute is

the noisy GPS C/A code but the high-precision Galileo E1C pilot. The attribute lookup order is `[system][attr]` → `[default][attr]` → 1.0. Code and phase have separate band and attribute tables (`SatBlockNoiseParams::ObservableScale`). Chip-rate calibration governs code, while carrier-tracking calibration governs phase and varies much less between bands. As with the other geometry strategies, the block variance is finally amplified by Equation (2). Phase reaches this path only when `apply_to_phase` is set. Otherwise it takes the flat `default_phase_std_m` route in the top-level dispatch.

All four strategies fill diagonal entries only, so the undifferenced $\boldsymbol{R}$ is block-diagonal, as in the observation noise model. Off-diagonal correlation appears only after differencing (Section 1.5).

1.2 A-posteriori variance factor

The Kalman-side QC layer of this section and the next three is a diagnostic that is *off by default*. `KalmanFilter::update` calls `computeResidualQc_` only when `estimation.enable_variance_factor` or `estimation.enable_data_snooping` is set. The correlation-aware kernel of Section 1.5 also requires `estimation.enable_full_r_kernel`. All three default to false. With them off, the only outlier gate is the fixed $|s| > 6$ path of the gating section, the variance factor stays at its initial value of one, and the redundancy numbers, w-tests and minimal detectable biases below are not computed.

After an epoch's update, the post-fit residuals $\hat{\boldsymbol{v}}$ show whether the assumed $\boldsymbol{R}$ was correctly scaled. The a-posteriori *variance factor* (estimated variance of unit weight) is

$$\boxed{\hat{\sigma}_0^2 = \frac{\Omega}{\mathrm{dof}}, \qquad \Omega = \hat{\boldsymbol{v}}^{\mathrm{T}} \boldsymbol{R}^{-1} \hat{\boldsymbol{v}} = \sum_i \frac{\hat{v}_i^{\,2}}{R_{ii}} \,\Big|_{\boldsymbol{R}\ \mathrm{diag}}, \qquad \mathrm{dof} = \mathrm{tr}\,\boldsymbol{\Theta},} \tag{6}$$

where Ω is the weighted sum of squared residuals and the degrees of freedom dof is the trace of the redundancy matrix $\boldsymbol{\Theta}$ (Equation (12) below). This is the *effective* redundancy of a recursively filtered epoch, not the $m - n$ of a single batch (`computeResidualQc_`, `src/core/filter/KalmanFilter.cpp`). $\boldsymbol{R}$ is in absolute units, so the a-priori variance of unit weight is $\sigma_0^2 = 1$. A well-scaled model gives $\hat{\sigma}_0^2 \approx 1$. $\hat{\sigma}_0^2 \gg 1$ means the assumed noise is too small (optimistic), and $\hat{\sigma}_0^2 \ll 1$ means it is too large (pessimistic).

The variance factor is used for *reporting only*. It never feeds back into the propagated covariance $\boldsymbol{P}$. When `enableVarianceFactor()` is set, the filter *logs* a copy of the position σ scaled by $\sqrt{\hat{\sigma}_0^2}$ and leaves the filter state unchanged (`validatePostFitResiduals`, `src/core/filter/KalmanFilter.cpp`).

The same quantity has a batch form in the normal-equation accumulator of the PPP NEQ-export path (constructed only under `enableNeqExport()` in `Application::run`). Given normal matrix $\boldsymbol{N}$, right-hand side $\boldsymbol{u}$, and the pre-reduced $\boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l}$ term, the accumulator forms (`NeqAccumulator::solve`)

$$\Omega = \boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l} - \boldsymbol{u}^{\mathrm{T}}\hat{\boldsymbol{x}}, \qquad \mathrm{dof} = n_{\mathrm{obs}} - (n_{\mathrm{params}} + n_{\mathrm{local}}), \tag{7}$$

$$\hat{\sigma}_0^2 = \frac{\Omega}{\mathrm{dof}}, \qquad \sigma_i = \sqrt{\hat{\sigma}_0^2 \left(\boldsymbol{N}^{-1}\right)_{ii}}. \tag{8}$$

Here $\hat{\boldsymbol{x}} = \boldsymbol{N}^{-1}\boldsymbol{u}$ is the least-squares solution. The redundancy subtracts the Schur-reduced local parameters n_{local} as well as the retained parameters n_{params}. The formal σ of each retained parameter is the square root of the corresponding diagonal of the inverse normal matrix, scaled by $\hat{\sigma}_0^2$. This is the batch counterpart of the recursive dof in Equation (6).

The network producer uses neither of these as its *operational* variance factor. The undifferenced network estimator does not form Ω/dof. It estimates a robust unit-weight σ as a scaled median absolute deviation over the weighted phase-leveling residual rows $\tilde{v}_i = |\hat{v}_i|\sqrt{w_i}$,

$$\hat{\sigma}_{0,\mathrm{net}} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \mathrm{median}_i\,\tilde{v}_i\big), \tag{9}$$

where the factor 1.4826 converts the MAD to a Gaussian σ and `raim_sigma0_floor` (default 1.0) is a lower bound. Unlike the Kalman variance factor of Equation (6), $\hat{\sigma}_{0,\mathrm{net}}$ is *not* used for reporting only. It controls three things:

- a within-epoch RAIM adaptation loop, whenever it exceeds `network.raim_sigma0_trigger`;
- a whole-epoch and a per-constellation product withhold, above `network.raim_withhold_sigma0`;
- the scale of the emitted URA.

Three separate seams implement them. `networkSolveGlue` (`filter/NetworkSolveGlue`) owns the within-epoch loop and both withhold gates, the per-constellation one through `NetworkEstimator::decidePerConstWithhold`. `networkPostFitRaim` (`filter/NetworkRaim`) owns the post-fit pass and the exponentially weighted fold of $\hat{\sigma}_{0,\mathrm{net}}$ into the emitted URA scale. `networkSolPrep` (`filter/NetworkSolPrep`) owns the emission-side prune that removes a withheld constellation's satellites. Sections 1.6 and 1.7 develop the adaptation loop and the pooled statistics. The emitted product is not treated here.

The network path has a third variance-factor instrument, which measures and does not act. `NetworkCarryVce` checks the hand-set carry-row stochastic model against the carry residuals $\hat{e}_i = \bar{b}_i - \hat{x}_i$ (the carried prior minus the solved value). Its primary output is a tail-robust geometric mean of $\hat{e}_i^2$, and a residual-form least-squares variance-component estimate of Floor and Rate components at fixed weight is reported beside it. The loader default is off (`network.carry_vce`). Both golden configurations arm it as a monitor of the ambiguity, phase-bias and code-bias carries, and in that role it never feeds back into the solve.

The feedback is `network.carry_vce_apply` (`filter/NetworkCarryVceApply`), built and default off. It multiplies the fold variance p_i of each carry family and constellation by a factor $\alpha^2 \geq 1$, updated by the variance-factor iteration of the LASER guide (slides 5-93 to 5-95) [3], $\alpha^2_{j+1} = \hat{\sigma}^2_j\,\alpha^2_j$. The group estimate $\hat{\sigma}^2$ divides the weighted residual square sum by the group's redundancy [2]. A carry row folded at $\alpha^2 p_i$ with posterior variance v_i has the partial redundancy $r_i = 1 - v_i/(\alpha^2 p_i)$ and $E\{\hat{e}_i^2\} = r_i\,\alpha^2 p_i\,\hat{\sigma}^2$. The streaming target is therefore formed row by row, as a geometric mean consistent with the monitor's:

$$\alpha^2 \ \leftarrow\ \frac{\exp\big(\overline{\log(\hat{e}_i^2/r_i)}\big)/c_{\chi^2_1}}{\exp\big(\overline{\log p_i}\big)}, \qquad c_{\chi^2_1} = e^{-\gamma}/2 \approx 0.2807, \tag{10}$$

where the overbar is the monitor's forgetting-weighted mean, p_i is the hand fold variance before the factor, and $c_{\chi^2_1}$ is the geometric mean of a χ^2_1 variate. A row with r_i at or below 10^{-3}, or with $r_i < 0$, carries no information on the factor and is left out and counted. The case $r_i < 0$ arises on the coupled ambiguity carry, whose harvested variance is a conditional variance that can exceed the prior; there r_i is an approximation. The factor is inflate-only, slew-limited, frozen on a contaminated epoch, and valid only from a fresh convergence. Until 2026-10-07 the target omitted the division by r_i. For a scalar carry with true prior variance $a\,p$ and data variance d, $E\{\hat{e}^2\} = r^2(ap + d)$, so that target settled below a, and once d exceeded about $ap/4$ it stayed at $\alpha^2 = 1$: the under-inflation was largest where the carry is stiff against its data.

1.3 Redundancy numbers, the Baarda w-test, and the MDB

The detection theory rests on the post-fit residual covariance $\boldsymbol{Q}$ (the covariance of $\hat{\boldsymbol{v}}$),

$$\boldsymbol{Q} \;=\; \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}^{+}\,\boldsymbol{H}^{\mathrm{T}}, \tag{11}$$

where $\boldsymbol{P}^{+}$ is the *updated* (post-fit) state covariance. `KalmanFilter::update` builds the diagonal $\boldsymbol{Q}_{ii}$ per row as $R_{ii} - (\boldsymbol{H}\boldsymbol{P}^{+}\boldsymbol{H}^{\mathrm{T}})_{ii}$. The full matrix is formed only under the off-diagonal kernel of Section 1.5.

Redundancy number. The (local) redundancy of observation i is

$$\boxed{r_i \;=\; \frac{\boldsymbol{Q}_{ii}}{R_{ii}} \;=\; \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i \in [0,1],} \tag{12}$$

clamped to $[0,1]$ for numerical safety in both branches of `KalmanFilter::residualQcKernel` (called by `computeResidualQc_`). When $r_i \to 1$ the geometry checks the observation fully, and a blunder appears almost entirely in the residual. When $r_i \to 0$ the observation is *uncheckable*: the estimate absorbs a blunder and the residual shows nothing. The sum dof $= \sum_i r_i = \operatorname{tr}\boldsymbol{\Theta}$ is the effective number of degrees of freedom used in Equation (6).

The network producer computes Equation (12) in the batch (arrowhead) algebra of the Schur-conditioned shared covariance, $r_i = 1 - w_i\,\boldsymbol{a}_i^{\mathrm{T}}\operatorname{Cov}(\boldsymbol{x})\,\boldsymbol{a}_i$ with $\boldsymbol{a}_i = [\,\boldsymbol{a}_{s,i} \mid \boldsymbol{a}_{r,i}\,]$ and the same $[0,1]$ clamp. This is algebraically the batch form of $r_i = \boldsymbol{Q}_{ii}/R_{ii}$ (`NetworkNormalEquations::rowRedundancy`, gated by `network.raim_redundancy_normalize`, default off). Rows below the `kRedFloor` redundancy floor are uncheckable and are dropped from the $\hat{\sigma}_{0,\mathrm{net}}$ pool of Equation (9). This corresponds to the `kUncheckableRedundancy` cutoff of the KF snooping gate below. The network constant is much smaller, because it protects only the $\sqrt{r_i}$ standardisation and not a reset decision.

One case of $r_i = 0$ is exact, and it determines the design of the network producer's phase-side QC. Suppose each (station, satellite, signal) arc has its own free ambiguity, which is the case whenever `network.phase_leveling` is off. Then every phase row is fitted *exactly* by its own parameter. The post-fit phase residual is identically zero, r_i is exactly zero, and the residuals cannot reveal a phase fault of any size. An undifferenced network with per-arc float ambiguities has *no* phase-side redundancy.

The phase rows gain redundancy, and so the power to detect faults, only when a carried prior constrains the ambiguities. That is why the producer's phase RAIM (Section 1.6) requires phase-leveling: without it the test has no power at all. The measure-only instrument `networkPostfitSigma0` (`filter/NetworkFrameDiag`) reports the effect. Its pooled phase variance factor is identically zero in that configuration, and the zero is the correct result. Its pools exclude two classes of row for related reasons. Rows that touch a shared spherical-harmonic column are pseudo-observations, not data (Section 1.8). The carry-coupling rows describe the carry, not the epoch's measurements.

w-test. Baarda's data-snooping statistic standardises each residual by its own post-fit standard deviation. The diagonal (uncorrelated) form is

$$w_i \;=\; \frac{\hat{v}_i}{\sqrt{\boldsymbol{Q}_{ii}}} \tag{13}$$

(the diagonal branch of `KalmanFilter::residualQcKernel`). The divisor is $\sqrt{\boldsymbol{Q}_{ii}}$, the *true* residual standard deviation, not $\sqrt{R_{ii}}$. With $\sigma_0^2 = 1$ known (every $\boldsymbol{R}$ here is absolute), this is the standardised

residual of [2], chapter 9; it becomes the studentised residual when an estimated $\hat{\sigma}_0$ replaces σ_0. Under the null hypothesis $w_i \sim \mathcal{N}(0,1)$. The s used by the fixed gate floors the denominator at R and is therefore conservative. When the observations are correlated, the weight matrix $\boldsymbol{R}^{-1}$ takes the place of $1/R_{ii}$ (the `use_full` branch of the same routine):

$$\boxed{w_i \;=\; \frac{\left(\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}\right)_i}{\sqrt{\left(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1}\right)_{ii}}}.} \tag{14}$$

This is Baarda's statistic for a single blunder in observation i [1, 6], the most powerful test of that alternative, and the form of the LASER guide's data snooping [3]. In a Kalman update $\boldsymbol{R}^{-1}\hat{\boldsymbol{v}} = \boldsymbol{S}^{-1}\boldsymbol{z}$ and $\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1} = \boldsymbol{S}^{-1}$, with $\boldsymbol{z}$ the innovation and $\boldsymbol{S}$ its covariance, so Equation (14) is also the innovation w-test $(\boldsymbol{S}^{-1}\boldsymbol{z})_i/\sqrt{(\boldsymbol{S}^{-1})_{ii}}$ [5, 7]. It reduces to Equation (13) when $\boldsymbol{R}$ is diagonal, although $\boldsymbol{Q}$ is not diagonal in a Kalman update. Until 2026-10-07 the branch used $\boldsymbol{Q}^{-1}$ in place of $\boldsymbol{R}^{-1}$. That statistic is also standard normal under the null hypothesis, but it tests the direction $\boldsymbol{S}\boldsymbol{R}^{-1}\boldsymbol{e}_i$ rather than observation i: it lost power against a single blunder, and it did not reduce to Equation (13) even for a diagonal $\boldsymbol{R}$. `tests/test_residual_qc_wtest.cpp` checks the routine against a batch Gauss–Markov model with the prior as stochastic constraints.

Minimal detectable bias. The smallest blunder on observation i that the w-test will detect, at the configured significance and a conventional 80% test power, is

$$\boxed{\mathrm{MDB}_i \;=\; \frac{\sigma_i\,\delta_0}{\sqrt{r_i}}, \qquad \delta_0 = \texttt{dataSnoopingCritical()} + 0.8416,} \tag{15}$$

with $\sigma_i = \sqrt{R_{ii}}$ the a-priori observation σ (the diagonal branch of `KalmanFilter::residualQcKernel`). With correlated observations the same definition gives $\mathrm{MDB}_i = \delta_0/\sqrt{(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1})_{ii}}$, so that a blunder of size MDB_i in observation i shifts the expectation of Equation (14) by exactly δ_0; it equals Equation (15) when $\boldsymbol{R}$ is diagonal. The non-centrality δ_0 is the critical value plus $\Phi^{-1}(0.80) = 0.8416$, for a test power of 80% ($\beta = 0.20$). For the default critical value $k = 3.29$ this gives $\delta_0 \approx 4.13$. The factor $1/\sqrt{r_i}$ expresses the geometry. A poorly checked observation ($r_i \to 0$) has an arbitrarily large MDB, and the code returns $+\infty$ when r_i is below `kRedFloor`. A blunder on such a row cannot be detected from the residual. The network producer computes the same quantity from its own redundancy numbers (`filter/NetworkMdb`, `network.mdb`), with $\sigma_i = 1/\sqrt{w_i}$. Both paths take δ_0, the redundancy floor and the $+\infty$ convention from one header, `filter/BaardaReliability.hpp`, so a PPP MDB and a network MDB are commensurable. The network MDB is measure-only: no gate, deweight or withhold reads it. Its loader default is off, and both golden configurations arm it. The producer's acting phase-side QC remains the robust $\hat{\sigma}_{0,\mathrm{net}}$ RAIM loop of Equation (9).

1.4 Data-snooping gate override

When `enableDataSnooping()` is set, the QC statistics drive an *alternative* outlier gate, which can select a row that the fixed $|s| > 6$ path missed. Both gates use the same downstream dispatch in `KalmanFilter::validatePostFitResiduals`. Only the *statistic, index, and critical value* differ. By default the gate is seeded from the fixed-σ triple $(\max_i |s_i|, \arg\max, 6)$, so the dispatch is byte-identical when snooping is off.

When snooping is enabled the gate is seeded from the w-test over *checkable* rows only. Rows with redundancy below the `kUncheckableRedundancy` cutoff are skipped. Their residuals are unreliable,

and resetting the corresponding ambiguity would discard good information. The same cutoff filters the summary log, so the log and the gate agree. Over the remaining rows the gate fires on

$$\max_{\{i\,:\,r_i \geq \texttt{kUncheckableRedundancy}\}} |w_i| \;>\; \texttt{dataSnoopingCritical()}, \tag{16}$$

and the checkable row with the largest $|w_i|$ becomes the handled index. w_i uses the true residual σ (Equation (13)), not the R-floored s. The gate can therefore catch a blunder on a high-leverage row whose floored s stayed below 6, and in that case it overrides the s choice. The downstream Melbourne–Wübbena tiebreaker still works in the σ domain. When the w-test selects a row that the σ cut missed, no σ candidate exists, and the w-selected index is handled directly. The fixed standardised-residual path is described in the gating section.

1.5 Correlation-aware full-$\boldsymbol{R}$ kernel

The diagonal forms above assume that $\boldsymbol{R}$ and $\boldsymbol{Q}$ are diagonal, which is true for undifferenced PPP. For BSSD and baseline-DD epochs the differencing operator $\boldsymbol{D}$ of the differencing transformations turns the diagonal undifferenced covariance into a *dense* one, $\boldsymbol{R}_{\mathrm{SD/DD}} = \boldsymbol{D}\,\boldsymbol{R}_{\mathrm{ZD}}\,\boldsymbol{D}^{\mathrm{T}}$. It contains the cross-covariance between observations that share a pivot or reference. The off-diagonal kernel (`estimation.enable_full_r_kernel`, default off) also requires that a dense $\boldsymbol{R}$ was supplied for the epoch, which `KalmanFilter::update` checks with `dense_r_supplied`. Under it the full residual covariance Equation (11) is built densely and symmetrised, and the QC quantities follow without approximation:

$$\boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i = \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \mathrm{dof} = \operatorname{tr}\boldsymbol{\Theta}, \tag{17}$$

$$\Omega = \hat{\boldsymbol{v}}^{\mathrm{T}}\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}, \qquad w_i = \frac{\big(\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}\big)_i}{\sqrt{(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1})_{ii}}}, \qquad \mathrm{MDB}_i = \frac{\delta_0}{\sqrt{(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1})_{ii}}}. \tag{18}$$

$\boldsymbol{R}^{-1}$ comes from an $\mathrm{LDL}^{\mathrm{T}}$ solve against the identity. An $\boldsymbol{R}$ that is not positive definite has no weight matrix, and the routine then falls back to the per-row forms. When $\boldsymbol{R}$ is diagonal, $\boldsymbol{\Theta}$, w and MDB equal the per-row quantities of Section 1.3, whatever the off-diagonals of $\boldsymbol{Q}$. The diagonal of the full $\boldsymbol{Q}$ equals the per-row $\boldsymbol{Q}_{ii}$, so the fixed $|s| > 6$ gate is *unchanged behind the flag*; under data snooping (Section 1.4) the gate reads the correlation-aware w_i. The flag only makes the statistics correct on differenced epochs, where the residuals are correlated. Its scope is narrow. The gain and the Joseph-form update already use a dense $\boldsymbol{R}$ whenever one is supplied, with or without the flag. The flag decides only whether the *post-fit residual* covariance is formed densely. The differencing section describes the corresponding correlation on the model side.

1.6 Producer-side RAIM: within-epoch adaptation

When $\hat{\sigma}_{0,\mathrm{net}}$ exceeds its trigger, the network producer chooses the *granularity* of its response from a robust two-way decomposition of the epoch's standardised residuals. Taking the largest standardised residual m_{ri} of each (station r, satellite i) carried-ambiguity cell as a sparse two-way table and applying a Tukey median polish,

$$m_{ri} \;\approx\; \mu \;+\; \alpha_r \;+\; \beta_i \;+\; \gamma_{ri}, \tag{19}$$

splits the contamination into a grand effect μ, a per-station row effect α_r, a per-satellite column effect β_i, and an isolated-arc interaction γ_{ri} (`medianPolish2Way`, `filter/NetworkRobustStats`). The polish sweeps medians, not means, so a minority of very large cells (the outliers) does not

distort the additive effects. A bad satellite, which raises a whole column, can then be told apart from a bad station, which raises a whole row. Isolated arc slips remain in the interaction term.

Each effect is a candidate culprit. To be eligible it must satisfy effect$/\hat{\sigma}_{0,\mathrm{net}} > k_{\mathrm{RAIM}}$ for the configured gate (`network.raim_k_sigma`). The eligible candidates are ranked with the narrowest response first: on equal score an arc ranks above a satellite, and a satellite above a station. The station branch is also reduced by a bias factor (`network.raim_station_drop_bias`, active under `network.raim_surgical_station`). Excluding a station discards good arcs with the bad, so it is chosen only when it dominates by that margin. A consensus rule avoids removing arcs one at a time. When at least `network.raim_min_stations` of one satellite's arcs exceed the gate in one iteration, the arc candidate is replaced by the whole satellite. A corrupted *shared* clock or phase bias is a network-wide fault, not a set of independent local ones.

The response distinguishes a bad *measurement* from a stale *parameter*. This distinction is the central design decision. A station culprit is excluded, because its data are bad and exclusion is the right response to bad data. A satellite or arc culprit is almost always a cycle slip. A slipped ambiguity does not need less weight; it needs a new *value*. The epoch is therefore solved again on a rebuilt normal-equation system with the culprit's ambiguities *refloated*. Their measurements keep full weight, so the local block stays positive definite and no singularity is introduced. Their carried prior is withheld, so the new solve can find the new value and absorb the slip locally. The loop repeats until $\hat{\sigma}_{0,\mathrm{net}}$ falls below the trigger or a small fixed number of passes is reached. Only the rare contaminated epochs pay for a second solve, and any failure reverts to the last good solution (`networkSolveGlue`).

`network.raim` and `network.raim_within_epoch` both default to false. *Both* tiers also require the phase-leveling carry to be warm: the within-epoch loop inside `networkSolveGlue`, and the post-fit pass at the `networkPostFitRaim` call site. With `network.phase_leveling` off there are no carried ambiguities to refloat. By the redundancy argument of Section 1.3 there is also no phase-side redundancy to test. The producer's RAIM then does nothing, as a consequence of the model and not of a policy choice.

1.7 Pooled variance factors and the per-constellation carve-out

Equation (9) pools only the carried-ambiguity *phase* rows. Two extensions widen or partition that set of rows without changing the estimator. Both are off by default.

Code and iono-constraint pools. Originally no tier screened the code rows or the iono-constraint rows that tie each line of sight to the spherical-harmonic field. Under `network.raim_we_code_iono` the same scaled-MAD estimator is applied to each of them over its own disjoint set of rows,

$$\hat{\sigma}_{0,\mathrm{code}} = 1.4826\ \mathrm{median}_{i\in\mathcal{C}}\,\tilde{v}_i, \qquad \hat{\sigma}_{0,\mathrm{iono}} = 1.4826\ \mathrm{median}_{i\in\mathcal{I}}\,\tilde{v}_i, \tag{20}$$

each with its own trigger. Every row belongs to exactly one of the three sets. Neither new pool has the floor of Equation (9), and the phase pool still drives the whole-epoch ceiling. The adaptation gains two responses suited to what these rows can diagnose: a station exclusion for the epoch, and a network-wide *deweighting* of one satellite's code rows. The second is a fallback and not a drop, in line with the project's preference for degrading a measurement over discarding it.

Per-constellation carve-out. When the global ceiling trips, the default response withholds the whole epoch's product. Under `network.per_const_withhold` the same pool is instead partitioned by the ambiguity's satellite constellation and each part judged on its own,

$$\hat{\sigma}_{0,c} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \mathrm{median}\{\tilde{v}_i : i \in \mathcal{P}_c\}\big), \tag{21}$$

over the same rows $\mathcal{P}_c$ that the global statistic pools. The statistic and the ceiling are the same, and only the set of rows is restricted. A constellation is published if and only if $\hat{\sigma}_{0,c}$ is below that ceiling *and* $|\mathcal{P}_c|$ meets a minimum row count. Too few rows cannot show that a constellation is clean inside a contaminated epoch (`NetworkEstimator::decidePerConstWithhold`).

This is an approximation, in two respects. The residuals in Equation (21) are evaluated at the *joint* solution. A contamination in one constellation perturbs the shared per-station nuisance parameters (receiver clock, tropospheric delay, slant ionosphere) and leaks into the residuals of another. Each row's redundancy is also its share of the joint system's redundancy, not of a separate per-constellation solve. $\hat{\sigma}_{0,c}$ therefore under-separates coupling between constellations. The outcome "every constellation individually clean while the global statistic tripped" is a consequence of that approximation. It is published with a warning and counted. The rank and observability gate keeps whole-epoch semantics. A rank deficiency invalidates the declared datum, so the product would be a meaningless least-norm split and no healthy subset exists.

1.8 Station-relative deweighting of the ionosphere constraint

A single-layer spherical-harmonic VTEC surface cannot represent the slant structure of individual lines of sight. Most LOS residuals against the fitted field exceed 3σ at any degree and any network size. The exceedance is a persistent background that is roughly common to all stations, so an *absolute* 3σ gate would engage every station and diagnose nothing. The difficult sites are equatorial-anomaly and scintillation-belt stations, whose true slant ionosphere the field cannot represent. They stand out only as a *relative* excess. The gate below is therefore robust and relative to the network (`NetworkIonoDeweightGate`).

Each station's per-epoch misfit statistic m_r is first smoothed by an exponentially-weighted moving average with forgetting factor $\phi = 2^{-1/W}$, i.e. a half-life of W epochs, then standardised against the network's own robust centre and scale. Writing $\mathcal{M}$ for the set of mature, non-statically-listed stations,

$$z_r \;=\; \frac{\bar{m}_r - \mathrm{med}}{1.4826\,\mathrm{MAD}}, \qquad \mathrm{med} = \mathrm{median}_{r' \in \mathcal{M}}\, \bar{m}_{r'}, \qquad \mathrm{MAD} = \mathrm{median}_{r' \in \mathcal{M}} \big|\bar{m}_{r'} - \mathrm{med}\big|. \tag{22}$$

Centring on the median removes the common background. The MAD scale tolerates a contaminated minority, with a breakdown point of 50%, which an absolute threshold does not have. The robust centre is formed only when enough mature stations are available. Statically listed stations are excluded from it, so a known bad site cannot mask the detection of others.

Detection is hysteretic: a station engages at $z_r > \sigma_{\mathrm{hi}}$ and releases at $z_r < \sigma_{\mathrm{lo}}$, with σ_{lo} forced below σ_{hi}. While engaged the target weight follows a smooth IGG-style ramp, unity at the release threshold and equal to a configured floor $w_{\min}$ at and above the engage threshold, quadratic between:

$$\boxed{w_r^* = w_{\min} + (1 - w_{\min})\,(1 - t)^2, \qquad t = \frac{\mathrm{clamp}\big(z_r,\, \sigma_{\mathrm{lo}},\, \sigma_{\mathrm{hi}}\big) - \sigma_{\mathrm{lo}}}{\sigma_{\mathrm{hi}} - \sigma_{\mathrm{lo}}}.} \tag{23}$$

The realised weight slews toward w_r^* by at most a fixed increment per clean epoch and is clamped to $[w_{\min}, 1]$. The slew limit stops the gate oscillating in response to its own effect on the field fit.

The gate acts only on $\boldsymbol{R}$. The factor w_r multiplies the *information* of that station's pseudo-observation rows between the spherical-harmonic field and the slant ionosphere, which is equivalent to $\sigma^2 \mapsto \sigma^2 / w_r$. Those rows form one joint pseudo-observation that couples the station's per-LOS slant ionosphere to the shared field in both directions, so one w_r relaxes both. As $w_r \to 0$ the station's ionosphere decouples from the field and is determined by its own geometry-free combination.

It stays observable and no rank deficiency appears. The station still contributes to the shared clocks, biases, datum graph and position. This is a narrower response than dropping a difficult station. It changes weights only: no state is removed and no datum is altered.

Two limits apply. Epochs with a datum change, including a pivot handover, are excluded. The gate is neither fed nor advanced on them, because their residual spikes come from the datum change and not from station misfit, and every weight is held across them. The adaptive path is off by default (`network.iono_constraint_deweight_adaptive`). With it off and the static station list empty, every w_r is one and the producer is byte-identical to a build without the gate. The gate is advanced once per epoch from the serial post-solve harvest in `networkShHealth` (`filter/NetworkShHealth`) and is read, const and free of races, during the parallel per-station assembly.

1.9 State-domain emission gates

Everything above tests *residuals* against $\boldsymbol{R}$. The producer also has two gates that test the *solved state* and act on the emitted product, not on the estimation. They are a different kind of test, and neither lives in the seam whose name suggests it.

Spherical-harmonic field health. Under `network.sh_health_gate` (default off) the solved VTEC coefficients are read back out of the shared solution, the dual-layer degree-zero terms are summed to a total mean VTEC, and that value is tested against a physically admissible band together with a ceiling on the RMS of all coefficients. An out-of-band epoch must repeat for a configured number of consecutive epochs before the gate engages, so a single-epoch excursion does not trip it. The verdict affects *emission only*. The solve, the carry stash and every QC tier continue to run while emission is withheld, so an era lasting hours does not expire the carry, as skipping the epochs would. If a coefficient lookup fails, the gate state is left unchanged; a release is never invented. The gate has no policy for release or degradation: a bad era withholds the product for its full duration. That is an open policy item, not a design conclusion. The verdict is formed in `networkSolveGlue`, the only seam before emission that holds the solved shared vector. `networkShHealth` (`filter/NetworkShHealth`) owns the health *probe*, the deweight fold of Section 1.8 and the carry variance-component update, but not this verdict.

Clock-runaway quarantine. Under `network.clock_runaway_check` (default off) the epoch's satellite clock solutions are grouped by constellation, and a constellation is dropped from the emission when the median of $|\text{clock} - \text{median}_{\text{all}}|$ exceeds a magnitude ceiling, or the median |drift| exceeds a rate ceiling. Like the carve-out of Section 1.7, this reduces the scope of the product and does not withhold all of it. The code documents one hazard and does not defend against it. The magnitude test is referenced to the global median over *all* clock satellites. If the runaway constellation supplies half or more of them, the comparison inverts: the healthy minority exceeds the ceiling and is quarantined, and the runaway passes. The degenerate single-constellation case is warned at configuration load. The quarantine is in `networkSolPrep` (`filter/NetworkSolPrep`). `networkClockQc` (`filter/NetworkClockQc`) is a different tier. It contains the two-state oscillator model with its normalised-innovation fault test, the decoupled emit-clock-rate estimator, and the per-satellite sanity monitor for carried clocks.

References

[1] W. Baarda. *A Testing Procedure for Use in Geodetic Networks*. Vol. 2. Publications on Geodesy, New Series 5. Delft, The Netherlands: Netherlands Geodetic Commission, 1968.

[2] K. Snow. *Adjustment Computations*. Lecture notes based on courses GS 650, GS 651 and GS 762 taught by B. Schaffrin at The Ohio State University. Jan. 2026.

[3] K. Snow, F. Yang, and A. Whitacre. *A Guide to LASER: LSA Libraries and Programs, Version 2.7*. Polaris Geospatial Services. Columbus, Ohio, Aug. 2024.

[4] J. Sanz Subirana, J. M. Juan Zornoza, and M. Hernández-Pajares. *GNSS Data Processing, Volume I: Fundamentals and Algorithms*. ESA TM-23/I. Noordwijk, The Netherlands: ESA Communications, 2013.

[5] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.

[6] P. J. G. Teunissen. *Adjustment Theory: An Introduction*. Series on Mathematical Geodesy and Positioning. Delft, The Netherlands: Delft University Press, 2000.

[7] P. J. G. Teunissen and O. Montenbruck, eds. *Springer Handbook of Global Navigation Satellite Systems*. Cham, Switzerland: Springer, 2017. ISBN: 978-3-319-42926-7. DOI: 10.1007/978-3-319-42928-1.